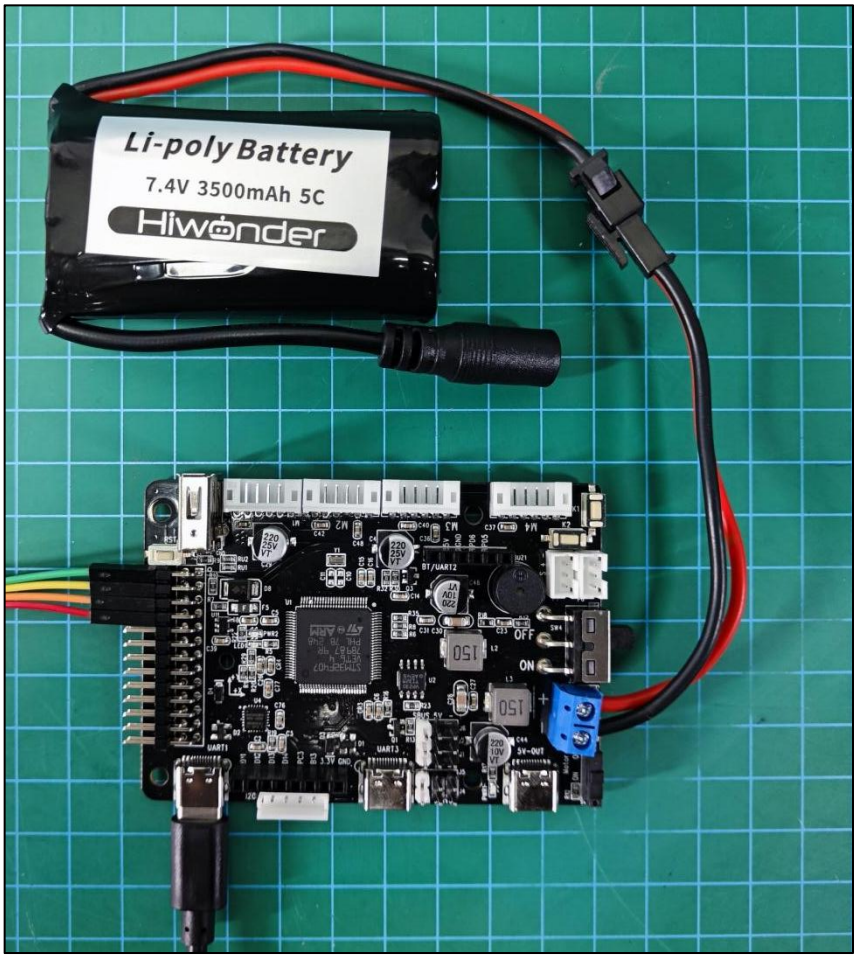
Lesson 4 RRC ADC Power Supply Voltage Detection

1. Program Description

This section primarily involves using the built-in ADC circuit in the STM32 main control board to read the battery voltage and output the voltage value via the serial port.

2. Hardware Connection

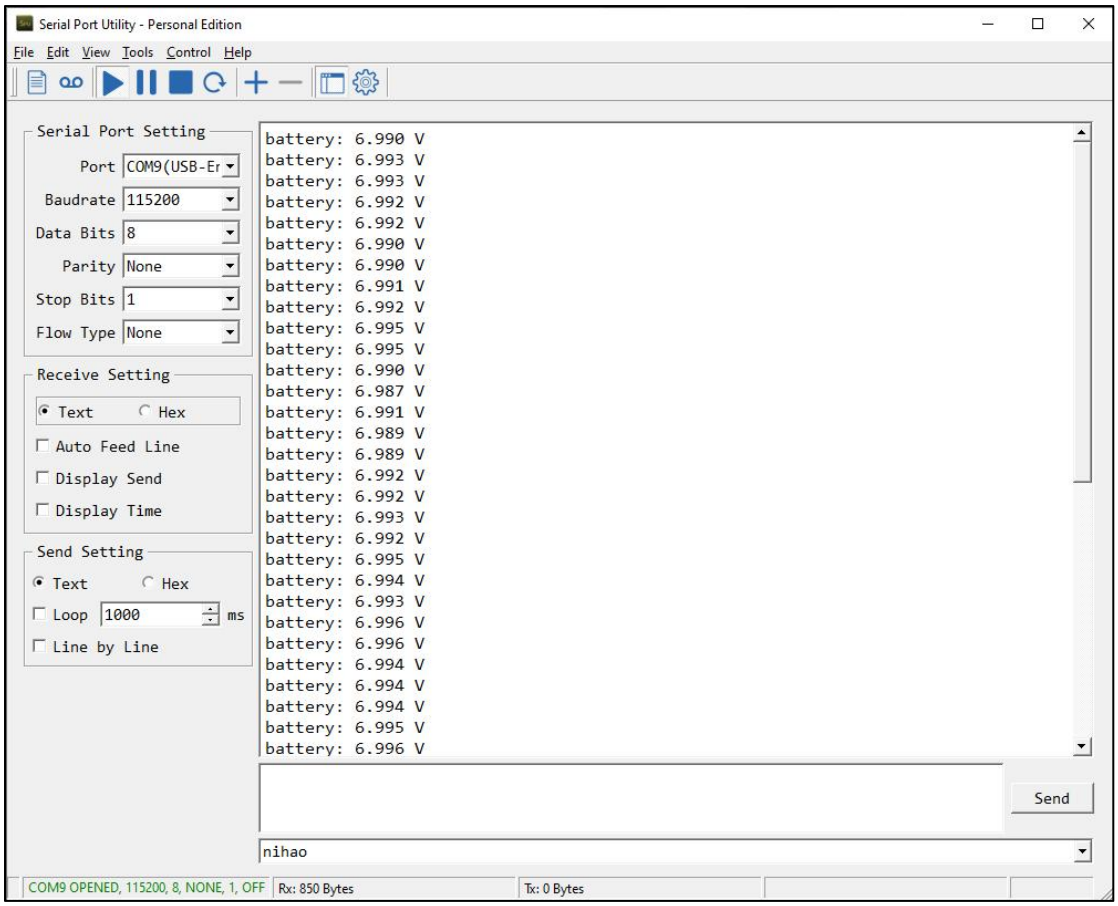
Connect the USB to serial port 1 and the other end to a PC. The hardware connection diagram is as follows:



3. Program Download

The example for this section is "rosrobotcontrollerm4_ADC_battery". For specific download methods, refer to the document: "2. STM32 Development Fundamentals/Lesson 3 Project Compilation and Download".

4. Program Outcome



Open the serial assistant (located in "Appendix/Software/Serial Assistant"), find the serial port connected to the development board (it may not necessarily be COM4, select based on actual conditions), and set it as follows:

Baud Rate: 115200

Data Bits: 8

Parity: None

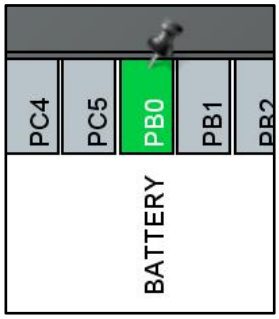
Stop Bits: 1

Display Encoding Format: UTF-8

After opening the serial port, you will see continuously refreshed data, where the data following "battery" is the battery voltage.

5. Pin Configuration

Use the STM32CubeMX software to view pin information. During program operation, the ADC conversion function of the pin will be enabled. The corresponding chip pin is: PB0



6. Source Code Analysis

The source code for this course is located in the same folder as this tutorial document, mainly analyzing the Hiwonder/System/app.c and Hiwonder/System/battery_handle.c files.

6.1 Voltage Print Output

app.c

```c
#include "cmsis_os2.h"
#include "lwmem_porting.h"
#include "global.h"
#include "adc.h"

/* 用户入口函数 */
void app_task_entry(void *argument)
{
    //声明外部变量
    //该变量是定义在battery_handle.c文件中的全局变量，存储ADC检测并转换后的电池电压
    extern float battery_volt;  /* 电池电压全局变量，单位 mV */

    /* 声明外部句柄 */
    //电量监控句柄
    extern osTimerId_t battery_check_timerHandle;

    //开启电量监控定时器，实时监控电量
    osTimerStart(battery_check_timerHandle, BATTERY_TASK_PERIOD);

    //开启ADC通道
    HAL_ADC_Start(&hadc1);

    //创建一个数据，用于存放字符串
    char msg[10] = "";

    // 循环 ：RTOS任务中的循环，必须要有osDelay或者其他系统阻塞函数，否则会导致系统异常
    for(;;) {

        //这里将浮点数转换为字符串之后再打印
        //因为单片机对浮点数的打印处理比较有限
        //为了保证系统的健壮性，将其先转换为字符串，再打印
        sprintf(msg , "%3.3f" , battery_volt/1000);

        printf("battery: %s V\n",msg);  //串口打印输出

        //延时500ms
        osDelay(500);

    }
}
```

The App.c file includes the user entry function app_task_entry(). The main programming flow is as follows:

Start the battery monitoring timer, which detects once every 50ms.

Then start the ADC conversion channel.

Finally, continuously print the battery voltage value.

The external variable battery_volt stores the measured battery voltage value. Therefore, by reading this value, you can obtain the voltage.

6.2 Reading Battery Voltage

battery_handle.c

```
13
14  void battery_check_timer_callback(void *argument)
15  {
16      if(adc_value[0] != 0 && adc_value[0] != 4095) { //过滤掉错误的判断
17          /* 1210.0是内部参考电压值， 100k + 10k 电阻分压， 实际电压是测量电压的11倍 */
18          float volt = 1210.0f / ((float)adc_value[0]) * ((float)adc_value[1]) * 11.0f ;
19          volt = volt > 20000 ? 0 : volt;     //判断是否符合实际电压（小车实际所用的电池电压不会高于20V）
20          battery_volt = battery_volt == 0 ? volt : battery_volt * 0.95f + volt * 0.05f;  //进行差值校准
21      }
22      HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_value, 2); //开启下一次的电压检测
23
```

The battery_handle.c file contains the battery_check_timer_callback() function, which is the callback function for the battery monitoring timer and is entered every 50ms to read the voltage value. The ADC value read will be stored in the adc_value array. The program flow is as follows:

First, check if adc_value[] is an error read value.

If the read value is correct, calculate and convert the value to a voltage value and store it in the global variable battery_volt.

If incorrect, do nothing.

Start a new ADC value reading.

The HAL_ADC_Start_DMA() function enables the DMA method for ADC reading:

HAL_StatusTypeDef HAL_ADC_Start_DMA (ADC_HandleTypeDef* hadc, uint32_t* pData , uint32_t Length)

Parameter 1: ADC handle

Parameter 2: Storage address of the ADC value

Parameter 3: Size of the ADC data

This function allows the ADC value to be automatically stored in the

storage address without needing the CPU.

This function allows the ADC value to be automatically stored in the storage address without needing the CPU.